Lightweight Knowledge Distillation for Multi-label Code Comment Classification

Muhammad Abdul Majeed
FAST-NUCES
Islamabad, Pakistan
i221216@nu.edu.pk

Ahmed Bin Asim
FAST-NUCES
Islamabad, Pakistan
i220949@nu.edu.pk

Abstract

We present a knowledge distillation approach for multi-label code comment classification that achieves competitive performance with high computational efficiency. Our method combines (1) semantic boosters—manually crafted synthetic examples targeting underrepresented categories, (2) standardized distillation loss preventing gradient domination, and (3) adaptive per-label weighting. Distilling CodeBERT to MiniLM, we achieve 82% parameter reduction while maintaining 95% performance on semantic categories. On NLBSE'26 datasets (Java, Python, Pharo), our approach achieves macro F1-score of 0.668 with 0.85s inference time and 770 GFLOPs, yielding a competition score of 0.74.

CCS Concepts

• **Software and its engineering** → **Software evolution**; • **Computing methodologies** → *Artificial intelligence.*

Keywords

code comments, knowledge distillation, multi-label classification

ACM Reference Format:
Muhammad Abdul Majeed and Ahmed Bin Asim. 2026. Lightweight Knowledge Distillation for Multi-label Code Comment Classification. In *5th International Workshop on Natural Language-based Software Engineering (NLBSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3786164.3794836

1 Introduction

Code comments serve as essential documentation in software development, providing insights into implementation details, design rationale, usage examples, and deprecation warnings. Automatically classifying these comments into functional categories enables numerous applications including documentation quality assessment, technical debt identification, and code review automation [4].

The NLBSE'26 Tool Competition [1] addresses multi-label code comment classification across three programming languages: Java (7 categories), Python (5 categories), and Pharo (6 categories). This task presents significant challenges: (1) **severe class imbalance**—semantic categories like "rational" and "Expand" comprise less than 5% of training samples, (2) **semantic complexity**—categories like "design rationale" require deep contextual understanding beyond surface patterns, and (3) **efficiency constraints**—models must achieve fast inference for real-time integration into development tools.

Recent approaches employ transformer-based models with promising results [2, 3]. However, these models face two critical limitations: computational overhead limiting real-time deployment, and poor performance on minority classes due to class imbalance. While STACC [3] addresses efficiency through sentence transformers (F1: 0.58-0.73), maintaining high performance on semantically complex, underrepresented categories remains challenging.

This paper introduces a knowledge distillation framework combining three innovations:

(1) **Semantic Booster Generation:** Manually crafted synthetic examples for underrepresented semantic categories, achieving up to 105% F1 improvement on minority classes
(2) **Standardized Knowledge Distillation:** Per-sample standardization preventing gradient domination, enabling stable convergence
(3) **Adaptive Semantic Weighting:** Label-wise weights emphasizing semantic categories during distillation

We distill CodeBERT (125M parameters) to MiniLM (22M parameters), achieving 82% parameter reduction while maintaining 95% teacher performance on semantic categories. Our model achieves macro F1-score of 0.668 with 0.85s inference time and 770 GFLOPs, yielding competition score 0.74.

2 Related Work

2.1 Code Comment Classification

Pascarella and Bacchelli [4] introduced a taxonomy for Java code comments, categorizing them into seven types (summary, expand, usage, deprecation, pointer, ownership, rational) using n-gram features and random forests. Their manually annotated dataset established baseline performance but struggled with semantic categories requiring contextual understanding.

Rani et al. [2] extended this to multi-language classification, developing classifiers for Java, Python, and Pharo class comments. Combining linguistic features with code structure analysis, they achieved macro F1-scores between 0.42-0.67. However, performance degraded significantly on semantically complex categories like "rational" and "intent," highlighting the challenge of capturing contextual meaning with traditional features.

Al-Kaswan et al. [3] proposed STACC, which fine-tunes sentence-BERT models on comment classification. While achieving improved F1-scores (0.58-0.73), their approach required substantial computational resources and struggled with severe class imbalance, particularly for categories comprising less than 5% of training data.

NLBSE '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2396-4/26/04
https://doi.org/10.1145/3786164.3794836

2.2 Knowledge Distillation

Knowledge distillation compresses large models by training smaller students to mimic teacher behavior. Standard MSE distillation directly matches logits but suffers from scale sensitivity—when teacher and student outputs differ in range, gradients become dominated by high-confidence predictions. Recent work in NLP has explored various distillation objectives, though multi-label scenarios with severe class imbalance remain underexplored.

In code understanding, CodeBERT and its derivatives have emerged as powerful pre-trained models. However, their 125M+ parameters limit deployment in latency-sensitive applications. Our work addresses this through standardized distillation while specifically targeting minority-class performance through semantic augmentation.

3 Methodology

Our approach combines targeted data augmentation, standardized knowledge distillation, and adaptive training to address class imbalance while maintaining efficiency.

3.1 Problem Formulation

Given training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ where x_i is a code comment and $y_i \in \{0, 1\}^C$ is a multi-label vector, we train a lightweight student model $f_S : \mathcal{X} \to \mathbb{R}^C$ to achieve high macro F1-score while meeting efficiency constraints: inference time < 5 seconds per test set and computational complexity < 5000 GFLOPs.

3.2 Semantic Booster Generation

3.2.1 Identifying Semantic-Hard Categories. We define **semantic-hard labels** as categories exhibiting both (1) low training frequency ($< 5\%$ prevalence) and (2) high semantic complexity requiring contextual understanding beyond surface lexical patterns. Through empirical analysis of training data distribution and error patterns, we identify:

- **Java:** Expand (4.1% prevalence—implementation details requiring deep code understanding), rational (3.2%—design justifications requiring architectural reasoning)
- **Python:** DevelopmentNotes (6.4%—internal TODOs and technical debt markers), Expand (8.7%)
- **Pharo:** Responsibilities (12.3%—component role descriptions), Intent (design purpose requiring understanding of class interactions), Collaborators (2.8%—inter-class relationship patterns)

These semantic-hard categories align with our booster targets, as they represent the primary classification challenge.

3.2.2 Manual Booster Authoring Process. We manually author semantic boosters following a template-based approach. Each booster is a synthetic code comment sentence designed to emphasize characteristic linguistic patterns of its target category. The authoring process:

(1) **Pattern Analysis:** We analyze training examples from each semantic-hard category to identify recurring linguistic markers (e.g., "The reason for..." in rational comments, "Note that..." in Expand comments)

Table 1: Manually Authored Semantic Booster Examples

Category	Example Text
Java Expand	"Note that this method assumes the input list is already sorted, otherwise undefined behavior may occur."
Java rational	"This design exists to minimize memory allocations during frequent calls."
Python Dev.	"TODO: Refactor this section once the new API stabilizes."
Pharo Intent	"Purpose: encapsulate retry logic to simplify error handling."

(2) **Template Creation:** We create sentence templates incorporating these markers with variable software engineering contexts
(3) **Manual Instantiation:** We manually instantiate each template with diverse technical content to ensure contextual variety

Each booster CSV file contains exactly **20 manually authored examples** per semantic-hard category in that language:

- Java: 40 total (20 Expand + 20 rational)
- Python: 40 total (20 DevelopmentNotes + 20 Expand)
- Pharo: 60 total (20 Responsibilities + 20 Intent + 20 Collaborators)

The CSV format contains two columns: `combo` (comment text) and `labels` (binary vector as Python list string). Each booster is assigned only its target semantic label (single-label examples). Table 1 shows representative examples.

3.2.3 Integration into Teacher Training. Semantic boosters are used exclusively during teacher fine-tuning via direct concatenation to the official training split. Specifically:

(1) We load the original NLBSE'26 training split for each language
(2) We load the corresponding booster CSV file
(3) We concatenate boosters to the training split, creating an augmented dataset
(4) The teacher is fine-tuned on this augmented dataset for all epochs

Boosters are **not** used in student training—the student trains only on the original distribution but learns from teacher logits computed on augmented data. This prevents student overfitting to synthetic patterns while benefiting from enhanced semantic representations in teacher predictions.

The ratio of boosters to original training examples:

- Java: 40 boosters / 10,672 original = 0.375%
- Python: 40 boosters / 3,182 original = 1.26%
- Pharo: 60 boosters / 2,191 original = 2.74%

During training, boosters are sampled with equal probability to original examples (standard uniform shuffling). They appear throughout all 12/25 epochs of teacher training, not just initially.

3.3 Standardized Knowledge Distillation

Standard MSE distillation directly matches logits:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N \cdot C} \sum_{i,j} (z_T^{ij} - z_S^{ij})^2 \tag{1}$$

This suffers from scale sensitivity. When z_T and z_S have different output ranges, gradients become dominated by high-confidence predictions, causing the student to ignore subtle semantic distinctions.

We propose **per-sample standardization** before computing MSE:

$$\tilde{z}_T^i = \frac{z_T^i - \mu(z_T^i)}{\sigma(z_T^i) + \epsilon}, \quad \tilde{z}_S^i = \frac{z_S^i - \mu(z_S^i)}{\sigma(z_S^i) + \epsilon} \tag{2}$$

$$\mathcal{L}_{\text{KD}} = \frac{1}{N \cdot C} \sum_{i,j} w_j \cdot (\tilde{z}_T^{ij} - \tilde{z}_S^{ij})^2 \tag{3}$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ are mean and standard deviation computed across C labels for each sample i. This normalization balances gradient magnitudes across predictions.

We introduce adaptive semantic weighting using the same semantic-hard categories identified for boosters: $w_j = 1.5$ for semantic-hard labels (Expand, rational, DevelopmentNotes, Responsibilities, Intent, Collaborators), $w_j = 1.0$ for all other labels. This 50% up-weighting guides the student to prioritize challenging categories.

The final student loss combines hard label supervision with weighted distillation:

$$\mathcal{L}_{\text{student}} = (1 - \alpha) \cdot \text{BCE}(f_S(x), y; w_{\text{pos}}) + \alpha \cdot \mathcal{L}_{\text{KD}} \tag{4}$$

The distillation coefficient $\alpha = 0.65$ was determined through validation tuning. We experimented with $\alpha \in \{0.4, 0.5, 0.65, 0.8\}$ and found 0.65 provided optimal balance. This value is fixed throughout all student training (not scheduled).

Positional weighting $w_{\text{pos}} = (N_{\text{neg}}/N_{\text{pos}})^{0.75}$ addresses base class imbalance. Label smoothing $\epsilon = 0.03$ is applied to hard labels.

3.4 Model Architecture and Training

Teacher Training: CodeBERT (microsoft/codebert-base, 125M parameters) fine-tuned on augmented datasets (original + boosters). Configuration: input length 128 tokens, batch size 16, learning rate 2e-5 (AdamW default), epochs 12 (Java) / 25 (Python, Pharo), FP16 mixed precision.

Student Training: sentence-transformers/all-MiniLM-L6-v2 (22M parameters, 82% reduction). Configuration: input length 64 tokens, batch size 32, learning rate 2e-5 (AdamW default), epochs 25 (Java) / 40 (Python) / 50 (Pharo), FP16 mixed precision, $\alpha = 0.65$ (fixed), semantic weight $w_j = 1.5$ for semantic-hard labels.

Threshold Optimization: Two-stage tuning on 12% validation split: (1) global search for best single threshold $t^* \in [0.2, 0.6]$ maximizing macro F1, (2) per-label refinement searching $t_j \in [t^* - 0.2, t^* + 0.2]$ for each label j greedily.

Table 2: Overall Performance Comparison

Method	Macro F1	Time (s)	GFLOPs	Score
Vanilla MiniLM	0.612	0.82	765	0.69
Standard Distill.	0.641	0.84	768	0.71
Ours	**0.668**	**0.85**	**770**	**0.74**
Teacher (CodeBERT)	0.703	3.42	2850	0.57

Table 3: Semantic Category F1-Scores

Category	Vanilla	Standard	Ours	Gain
Java Expand	0.167	0.298	0.342	+105%
Java rational	0.201	0.289	0.325	+62%
Python DevNotes	0.289	0.401	0.455	+57%
Python Expand	0.412	0.518	0.554	+35%
Pharo Resp.	0.412	0.556	0.608	+48%
Pharo Collab.	0.051	0.089	0.143	+180%
Avg. Semantic	**0.255**	**0.359**	**0.405**	**+59%**

4 Experimental Results

4.1 Datasets and Evaluation

We evaluate on NLBSE'26 benchmark [1]: Java (10,672 train, 1,201 test, 7 categories), Python (3,182 train, 290 test, 5 categories), Pharo (2,191 train, 208 test, 6 categories). Metrics: macro F1-score, per-category F1, inference time, GFLOPs, and competition score: $0.6 \times \text{F1} + 0.2 \times \text{Efficiency}_{\text{time}} + 0.2 \times \text{Efficiency}_{\text{FLOPs}}$.

4.2 Overall Performance

Table 2 presents overall results. Our approach achieves macro F1 of 0.668, representing 95% of teacher performance (0.703) while requiring only 18% of parameters. Compared to vanilla training, distillation improves F1 by +0.056 (9.1% relative). Standardized distillation contributes +0.027 over standard MSE, demonstrating effectiveness of per-sample normalization. The 4× speedup (0.85s vs 3.42s) and 82% parameter reduction yield competition score 0.74.

4.3 Semantic Category Impact

Table 3 quantifies semantic category improvements. Semantic boosters achieve +59% average gain over vanilla training. Java's "Expand" improves 105% (0.167→0.342), demonstrating effectiveness of targeted augmentation. Pharo's "Collaborators" shows 180% improvement (0.051→0.143), though absolute performance remains challenging due to extreme rarity (2.8% prevalence). Standard distillation achieves 0.359, confirming boosters provide benefits beyond knowledge transfer alone.

4.4 Per-Language Analysis

Performance varies by language: Java achieves F1 0.732 (95.3% teacher retention), benefiting from larger training set. Python shows 0.641 (94.0% retention) despite smallest training data, suggesting concentrated booster effect (1.26% augmentation ratio). Pharo presents most difficulty (0.617, 93.6% retention) due to six semantic categories with complex collaborative relationships. All languages maintain consistent 94-95% knowledge retention.

Table 4: Ablation Study Results

Configuration	Macro F1	Δ F1
Full Model (Ours)	0.668	–
- Standardization	0.641	-0.027
- Semantic Weight	0.649	-0.019
- Semantic Boosters	0.630	-0.038
- All Distillation	0.612	-0.056

4.5 Ablation Study

Table 4 isolates component contributions. Removing standardization (-0.027 F1) causes gradient imbalance. Semantic weighting removal (-0.019 F1) reduces focus on challenging categories. Without semantic boosters, performance drops -0.038 F1. Eliminating all distillation (-0.056 F1) demonstrates knowledge transfer value.

4.6 Per-Category Performance

Our approach achieves competitive performance across categories. Strengths include Java Ownership (0.949), Pointer (0.910), summary (0.893); Python Usage (0.768), Parameters (0.761); Pharo Intent (0.884), Example (0.880). Challenges remain for extreme imbalance: Java rational (0.325), Pharo Collaborators (0.143) despite 180% improvement.

4.7 Efficiency Analysis

Memory consumption decreases from 1.8GB (teacher) to 0.4GB (student). The 0.85s inference processes 1,700 comments/second, sufficient for real-time IDE integration. GFLOPs reduction (2850→770, 3.7×) ensures computational efficiency.

5 Discussion

5.1 Key Findings

Our results demonstrate three key findings. First, targeted augmentation outperforms generic methods—by addressing semantic-hard categories, boosters achieve +59% average improvement. Second, standardization stabilizes distillation (+0.027 F1), enabling learning from subtle predictions. Third, efficiency need not compromise minority-class performance (95% retention with 82% compression).

5.2 Practical Implications

The 0.85s inference supports real-time documentation analysis. The 22M parameter count and 0.4GB memory enable direct IDE deployment. Multi-language support suggests applicability to additional languages.

5.3 Limitations

Manual booster authoring requires expert effort. Comment classification involves subjective judgments. Evaluation is limited to NLBSE'26 benchmark. Future work: automated booster generation, multi-task learning, cross-language transfer.

6 Conclusion

We presented a knowledge distillation framework achieving competitive performance (F1 0.668, score 0.74) with substantial efficiency gains. Semantic boosters, standardized distillation, and adaptive weighting enable lightweight models to maintain performance on challenging categories. The key insight: targeted minority-class intervention with calibrated knowledge transfer enables compression without sacrificing semantic understanding.

Acknowledgments

We thank the NLBSE'26 organizers and the HuggingFace/PyTorch communities.

References

[1] Moritz Mock, Pooja Rani, Fabio Santos, Benjamin Carter, and Jacob Penney. The NLBSE'26 Tool Competition. In *Proceedings of The 5th International Workshop on Natural Language-based Software Engineering (NLBSE'26)*, 2026.

[2] Pooja Rani, Sebastiano Panichella, Manuel Leuenberger, Andrea Di Sorbo, and Oscar Nierstrasz. How to identify class comment types? A multi-language approach for class comment classification. *Journal of Systems and Software*, 181:111047, 2021.

[3] Ali Al-Kaswan, Maliheh Izadi, and Arie Van Deursen. STACC: Code Comment Classification using SentenceTransformers. In *2023 IEEE/ACM 2nd International Workshop on Natural Language-Based Software Engineering (NLBSE)*, pages 28–31, 2023.

[4] Luca Pascarella and Alberto Bacchelli. Classifying code comments in Java open-source software systems. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, 2017.